Experiment No. 6

1. Dataset Source:

The dataset used in this experiment is the Calories Burnt Prediction Dataset.

- Source: Calories Burnt Dataset
- Link: https://www.kaggle.com/datasets/fmendes/fmendesdat263xdemos.

2. Dataset Description:

The dataset contains information related to physical activity and calories burned.

1. **Features:**
    - **User_ID** – Unique identifier of the user
    - **Age** – Age of the person
    - **Height** – Height (in cm)
    - **Weight** – Weight (in kg)
    - **Duration** – Exercise duration (minutes)
    - **Heart_Rate** – Heart rate during exercise
    - **Body_Temp** – Body temperature (°C)
    - **Calories** – Calories burned
2. **Target Variable:**
    - There is **no target variable** (unsupervised learning)
    - Goal: **Group similar data points into clusters.**
3. **Dataset Characteristics:**
    - Contains numerical features
    - Medium-sized dataset
    - No labels → suitable for clustering.

3. Mathematical Formulation of the Algorithm:

1. K-Means Clustering:

K-Means aims to minimize the **within-cluster sum of squares (WCSS)**:

$$J = \sum_{i=1}^{k} \sum_{x \in C_i} ||x - \mu_i||^2$$

Where:

- kkk = number of clusters
- CiC_iCi = cluster
- μi\mu_μi = centroid of cluster.

Steps:

1. Initialize centroids
2. Assign points to nearest centroid
3. Update centroids
4. Repeat until convergence

2. Hierarchical Clustering (Ward Method):

Distance between clusters is minimized using:

$$D(A, B) = \frac{|A||B|}{|A| + |B|} ||\mu_A - \mu_B||^2$$

It builds clusters in a bottom-up approach using a dendrogram.

4. Algorithm Limitations:

1. **K-Means:**
   - Needs predefined k value
   - Sensitive to outliers
   - Works best with spherical clusters
   - Results depend on initial centroids.
2. **Hierarchical Clustering:**
   - Computationally expensive for large datasets
   - Difficult to choose correct number of clusters
   - Cannot undo merging once done.

5. Methodology / Workflow:

Steps followed in experiment:

1. **Data Loading:**
   a) Dataset loaded using pandas
2. **Feature Selection:**
   b) Selected numerical features: Age, Height, Weight, etc.
3. **Handling Missing Values:**
   c) Replaced missing values using **mean**
4. **Feature Scaling:**
   a) Applied **StandardScaler** to normalize data.

5. **K-Means Clustering:**
    a) Used **Elbow Method** to find optimal k
    b) Applied K-Means with k = 3
6. **Hierarchical Clustering:**
    a) Created **Dendrogram**
    b) Applied Agglomerative clustering (k = 3)
7. **Visualization:**
    a) Scatter plots (Age vs Calories, Duration vs Heart Rate)
    b) Dendrogram.

6. Performance Analysis:

Since clustering is **unsupervised**, performance is evaluated using:

1. **Observations:**
    - Elbow method shows optimal k ≈ 3
    - Clusters are reasonably separated
    - Scatter plots show meaningful grouping
2. **Cluster Insights:**
    - Some clusters represent:
    - High calorie burn (high duration & heart rate)
    - Medium activity
    - Low activity

Both algorithms produced **similar cluster distributions.**

7. Hyperparameter Tuning:

1. **K-Means:**
   - n_clusters (k) → tuned using Elbow Method
   - init → k-means++ (better centroid initialization)
   - n_init → 10 (avoids poor solutions)

   Impact:

   - Better cluster stability
   - Reduced error (WCSS)

2. **Hierarchical Clustering:**
   - n_clusters → selected from dendrogram
   - linkage = ward → minimizes variance
   - metric = euclidean

   Impact:

   - More compact clusters
   - Better hierarchical structure.

CODE:

```
import pandas as pd

import matplotlib.pyplot as plt

import seaborn as sns

from sklearn.preprocessing import StandardScaler

from sklearn.cluster import KMeans, AgglomerativeClustering

from scipy.cluster.hierarchy import dendrogram, linkage

# --- 1. Data Loading and Initial Inspection ---

print("--- Data Loading and Initial Inspection ---")
```

```
df = pd.read_csv('/content/calories.csv')
print("Original DataFrame head:")
display(df.head())
print("\nDataFrame Info:")
df.info()
# --- 2. Feature Selection and Preprocessing ---
print("\n--- Feature Selection and Preprocessing ---")
features = ['Age', 'Height', 'Weight', 'Duration', 'Heart_Rate', 'Body_Temp', 'Calories']
X = df[features].copy()
print("Missing values before handling:")
display(X.isnull().sum())
X.fillna(X.mean(), inplace=True)
print("\nMissing values after handling:")
display(X.isnull().sum())
scaler = StandardScaler()
X_scaled = scaler.fit_transform(X)
print("\nScaled data head (first 5 rows and columns):")
display(pd.DataFrame(X_scaled, columns=features).head())
# --- 3. K-Means Clustering ---
print("\n--- K-Means Clustering ---")
# Elbow Method to find optimal k for K-Means
wcss = []
for i in range(1, 11):
    kmeans = KMeans(n_clusters=i, init='k-means++', random_state=42, n_init=10)
    kmeans.fit(X_scaled)
```

```
    wcss.append(kmeans.inertia_)

fig = plt.figure(figsize=(10, 6))

plt.plot(range(1, 11), wcss, marker='o')

plt.title('Elbow Method for K-Means')

plt.xlabel('Number of clusters (k)')

plt.ylabel('WCSS')

plt.grid(True)

plt.show()

# Apply K-Means clustering with k=3 (based on elbow method observation)

kmeans_model = KMeans(n_clusters=3, init='k-means++', random_state=42, n_init=10)

df['KMeans_Cluster'] = kmeans_model.fit_predict(X_scaled)

print("\nK-Means Cluster distribution (k=3):")

display(df['KMeans_Cluster'].value_counts())

# Visualize K-Means clusters

fig = plt.figure(figsize=(10, 7))

sns.scatterplot(x='Age', y='Calories', hue='KMeans_Cluster', data=df, palette='viridis', s=100,
alpha=0.7)

plt.title('K-Means Clustering: Age vs Calories')

plt.xlabel('Age')

plt.ylabel('Calories Burned')

plt.grid(True)

plt.show()

fig = plt.figure(figsize=(10, 7))

sns.scatterplot(x='Duration', y='Heart_Rate', hue='KMeans_Cluster', data=df, palette='viridis',
s=100, alpha=0.7)

plt.title('K-Means Clustering: Duration vs Heart Rate')
```

```
plt.xlabel('Duration (minutes)')

plt.ylabel('Heart Rate')

plt.grid(True)

plt.show()

# --- 4. Hierarchical Clustering ---

print("\n--- Hierarchical Clustering ---")

# Create Dendrogram to determine optimal number of clusters

linked = linkage(X_scaled, method='ward')

fig = plt.figure(figsize=(15, 8))

dendrogram(linked,

        orientation='top',

        distance_sort='descending',

        show_leaf_counts=True)

plt.title('Dendrogram for Hierarchical Clustering')

plt.xlabel('Sample Index')

plt.ylabel('Distance')

plt.show()

# Apply Agglomerative Clustering with 3 clusters (based on dendrogram observation)

hierarchical_model = AgglomerativeClustering(n_clusters=3, metric='euclidean', linkage='ward')

df['Hierarchical_Cluster'] = hierarchical_model.fit_predict(X_scaled)

print("\nHierarchical Cluster distribution (k=3):")

display(df['Hierarchical_Cluster'].value_counts())

# Visualize Hierarchical clusters

fig = plt.figure(figsize=(10, 7))

sns.scatterplot(x='Age', y='Calories', hue='Hierarchical_Cluster', data=df, palette='viridis', s=100,
alpha=0.7)
```

```
plt.title('Hierarchical Clustering: Age vs Calories')

plt.xlabel('Age')

plt.ylabel('Calories Burned')

plt.grid(True)

plt.show()

fig = plt.figure(figsize=(10, 7))

sns.scatterplot(x='Duration', y='Heart_Rate', hue='Hierarchical_Cluster', data=df,
palette='viridis', s=100, alpha=0.7)

plt.title('Hierarchical Clustering: Duration vs Heart Rate')

plt.xlabel('Duration (minutes)')

plt.ylabel('Heart Rate')

plt.grid(True)

plt.show()
```

OUTPUT:

```
Original DataFrame head:
```

	User_ID	Gender	Age	Height	Weight	Duration	Heart_Rate	Body_Temp	Calories
0	14733363	male	68	190.0	94.0	29.0	105.0	40.8	231.0
1	14861698	female	20	166.0	60.0	14.0	94.0	40.3	66.0
2	11179863	male	69	179.0	79.0	5.0	88.0	38.7	26.0
3	16180408	female	34	179.0	71.0	13.0	100.0	40.5	71.0
4	17771927	female	27	154.0	58.0	10.0	81.0	39.8	35.0

```
DataFrame Info:
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 15000 entries, 0 to 14999
Data columns (total 9 columns):
 #   Column      Non-Null Count  Dtype
---  ------      --------------  -----
 0   User_ID     15000 non-null  int64
 1   Gender      15000 non-null  object
 2   Age         15000 non-null  int64
 3   Height      15000 non-null  float64
 4   Weight      15000 non-null  float64
 5   Duration    15000 non-null  float64
 6   Heart_Rate  15000 non-null  float64
 7   Body_Temp   15000 non-null  float64
 8   Calories    15000 non-null  float64
dtypes: float64(6), int64(2), object(1)
memory usage: 1.0+ MB
```

```
Missing values after handling:
```

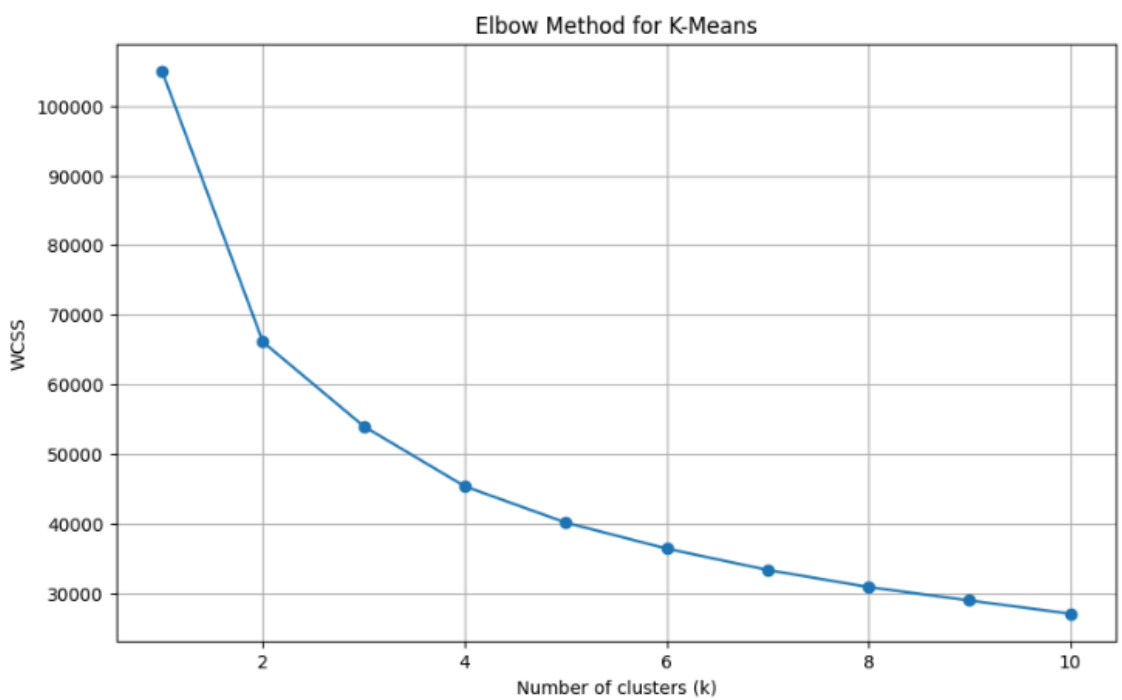
	0
Age	0
Height	0
Weight	0
Duration	0
Heart_Rate	0
Body_Temp	0
Calories	0

dtype: int64

```
Scaled data head (first 5 rows and columns):
```

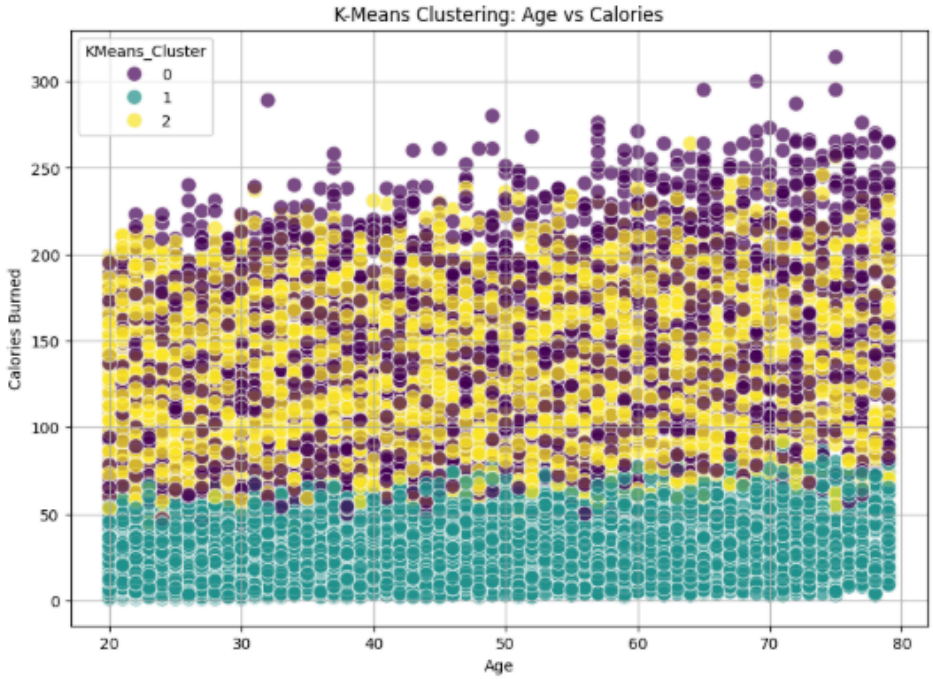
	Age	Height	Weight	Duration	Heart_Rate	Body_Temp	Calories
0	1.484726	1.089582	1.265909	1.619127	0.989404	0.994023	2.265002
1	-1.342179	-0.593726	-0.995458	-0.183990	-0.158461	0.352342	-0.376905
2	1.543620	0.318066	0.268247	-1.265861	-0.784569	-1.701035	-1.017367
3	-0.517665	0.318066	-0.263839	-0.304198	0.467647	0.609015	-0.296847
4	-0.929922	-1.435380	-1.128480	-0.664821	-1.515029	-0.289338	-0.873263

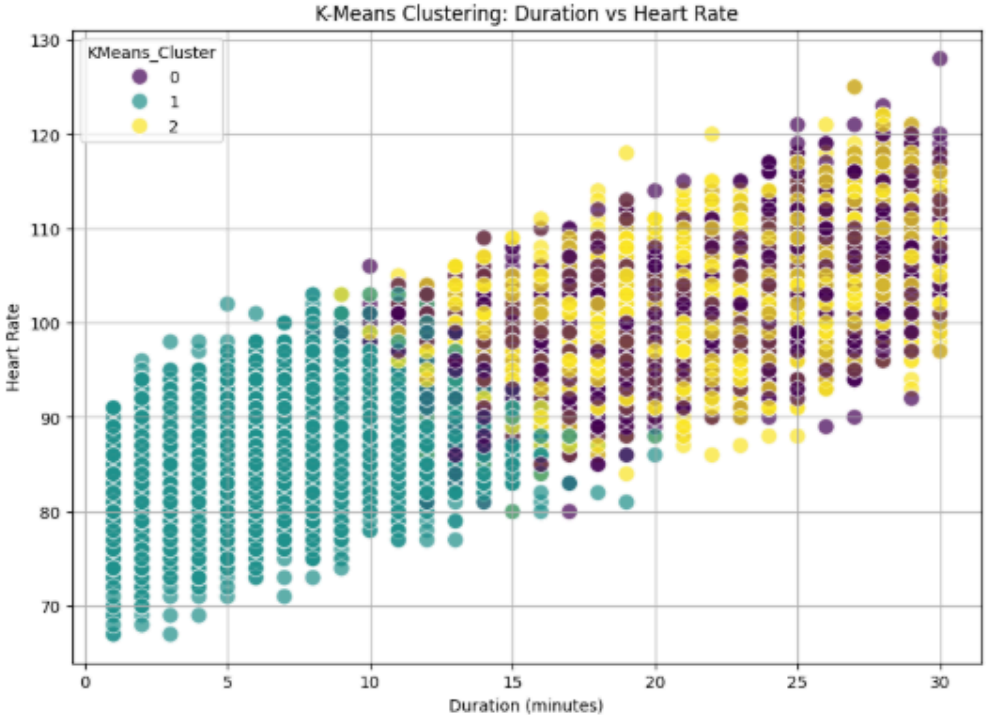
```
--- K-Means Clustering ---
```



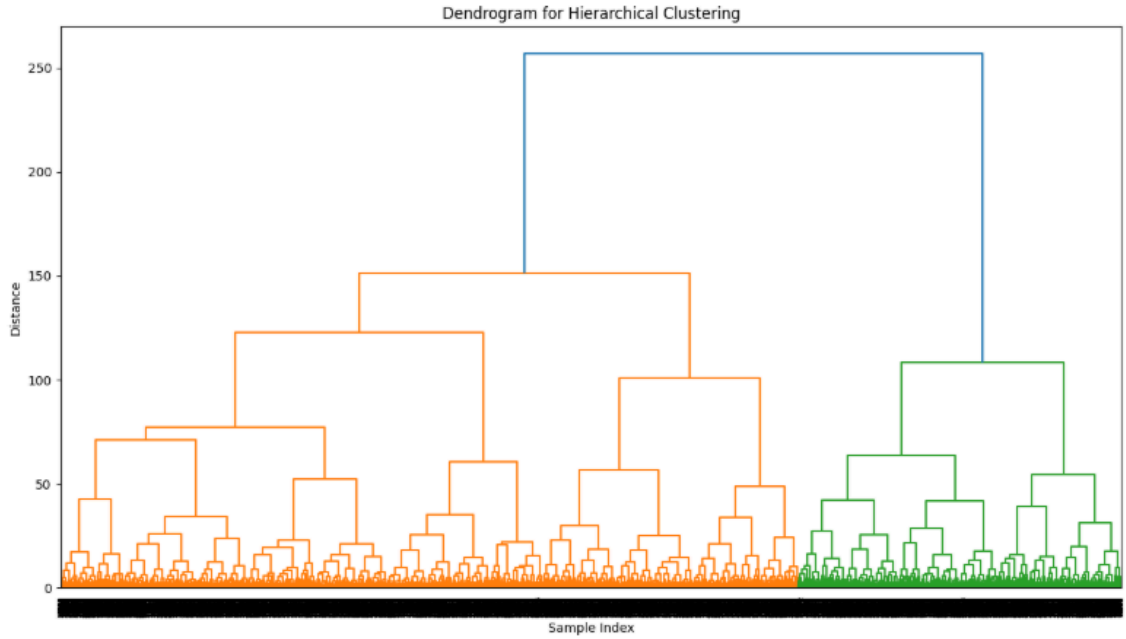
KMeans_Cluster	
1	6121
2	4630
0	4249

dtype: int64





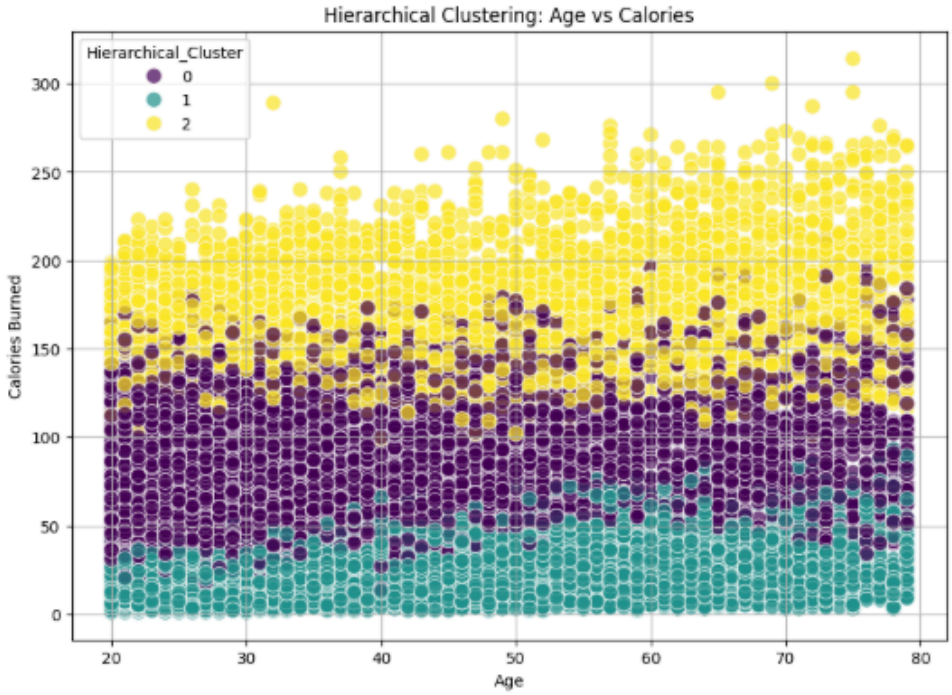
```
--- Hierarchical Clustering ---
```

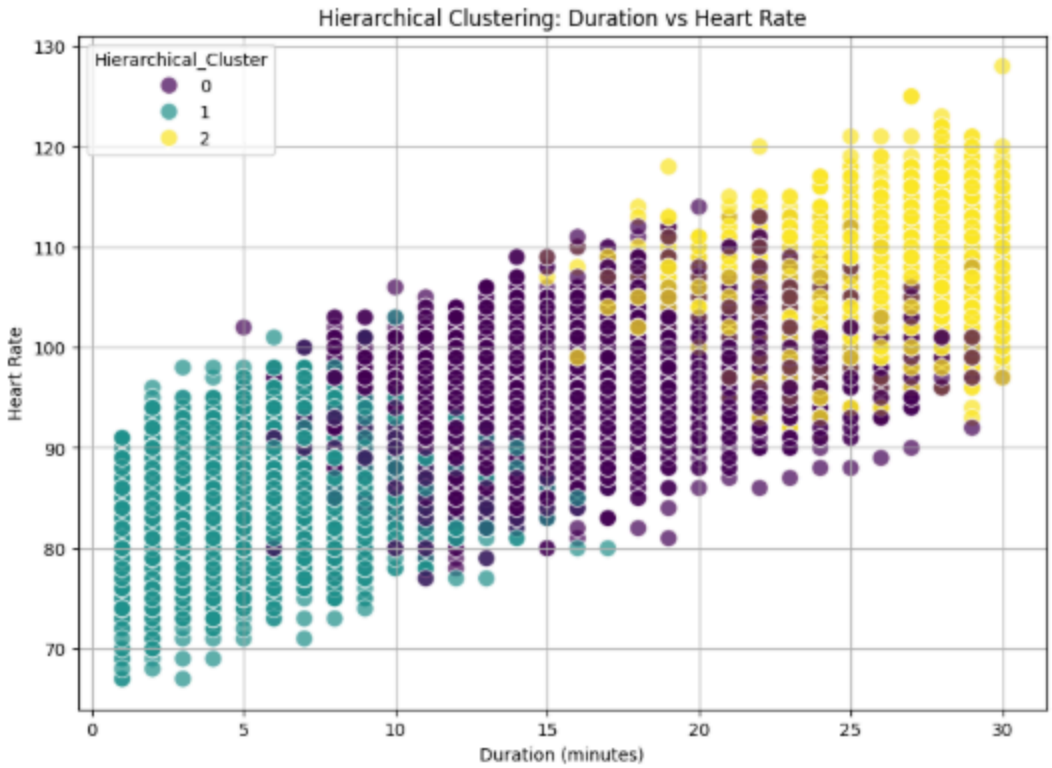



```
Hierarchical Cluster distribution (k=3):
```

Hierarchical_Cluster	count
0	6817
1	4573
2	3810

dtype: int64




Hierarchical Clustering: Duration vs Heart Rate
Hierarchical_Cluster
0
1
2
Heart Rate
Duration (minutes)
130
120
110
100
90
80
70
0
5
10
15
20
25
30